JOURNAL REPORT – AI Agent System Implementation

Applied System Software (ASD)

Student Name: Herath Mudiyanselage Udesha Uditha Kumara Herath

Student ID: 221AMB073

Step 1 – System Planning and Design

1. System Description and Goal

The goal of this project is to design and implement an AI-assisted software system using Python and Google Gemini API. The system functions as a command-line AI assistant capable of understanding user input, processing requests and executing tasks using external tools.

The main objective is to demonstrate how modern AI agents can combine reasoning capabilities with tool-based execution to solve practical problems.

2. AI / Agent-Based Approach

The system follows an AI-agent architecture where the Gemini model acts as the reasoning engine. The agent receives user input, analyzes the request, decides whether a tool is needed, executes the tool if required and then generates a final response.

The system implements a simplified ReAct (Reason → Act → Observe) workflow:

- Reason: Analyze user request
- Act: Execute tool if needed
- Observe: Process tool output
- Respond: Generate final answer

3. List of Tools

The system includes the following tools. These tools allow the agent to perform real-world tasks beyond simple text generation.

- Calculator Tool – performs mathematical operations
- File Reader Tool – reads and processes text files
- Time Tool – retrieves current system time
- Translator Tool – translates text between languages

4. Programming Concepts (Planned)

The following programming concepts were planned:

- Object-Oriented Programming (OOP)
- Modular system design
- Separation of concerns
- API integration
- Tool-based architecture
- Command-line interface development
- Error handling

Step 2 – Implementation Progress

1. Updated System Description

The system has been successfully implemented as a modular AI-agent application. It integrates the Gemini API and dynamically executes tools based on user input. The system is fully functional and capable of performing multiple operations through its tool-based architecture.

2. Programming Concepts Used

- Object-Oriented Programming (OOP)
- Modular architecture
- Tool Registry pattern
- ReAct workflow
- Dynamic tool invocation
- API integration
- Error handling
- Command-line interface

3. Application of Concepts

- OOP was used to structure the system into classes such as Agent, ToolRegistry and BaseTool.
- Modular design separates logic into independent components.
- Tool Registry allows dynamic tool management without modifying core logic.
- ReAct workflow controls how the agent processes requests and executes tools.
- Error handling ensures the system does not crash during invalid operations.

4. Tool Integration

The tools are integrated through a centralized Tool Registry. This design improves flexibility and scalability.

Workflow:

1. User enters a request
2. Agent analyzes the request
3. Agent selects appropriate tool
4. Tool executes the operation
5. Result is returned
6. Final response is generated

Step 3 – Testing and Deployment

1. Testing Process

Testing was performed using Python's built-in unittest framework. Testing was done continuously during development and includs:

- unit testing individual tools
- integration testing between components
- validation of error handling
- functional testing of system behavior

2. Test Scenarios

1. Calculator Tool
2. addition, subtraction, multiplication, division
3. division by zero handling
4. File Reader Tool
5. reading an existing file
6. handling non-existent files
7. Tool Registry
8. registering tools
9. retrieving tools
10. handling invalid tool requests

3. Error Handling Testing

The system was tested against:

- invalid mathematical expressions
- division by zero
- missing input parameters
- file not found errors
- unknown tool requests

The system was improved to return meaningful error messages instead of crashing.

4. Test Results

Total tests executed: 10

Passed: 10

Failed: 0

All components performed correctly under normal and edge-case conditions.

5. Deployment Preparation

The system is prepared for controlled execution:

- dependencies managed using requirements.txt
- modular project structure
- command-line execution
- How to run the system:

pip install -r requirements.txt

python main.py

6. Data Conversion and Handling

The system uses structured data (Python dictionaries) for communication between components. Each tool validates and processes data to maintain consistency and correctness.

For example:

- tool inputs (expression, filename)
- tool outputs (results returned as strings)